

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



HỌC PHẦN MỞ RỘNG MÔN CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

ĐỀ TÀI HỌC PHẦN MỞ RỘNG

TĂNG TỐC TÌM KIẾM GẦN ĐÚNG BẰNG
THUẬT TOÁN SCANN (SCALABLE NEAREST
NEIGHBORS)

LỚP: TN01

Giảng viên hướng dẫn: LÊ THÀNH SÁCH
Danh sách nhóm:

STT	Họ và tên	MSSV	Ghi chú
1	Huỳnh Hoàng Anh	2410078	
2	Ngô Trung Tín	2413506	
3	Huỳnh Tấn Tiến	2413468	

MỤC LỤC

1	Mục tiêu	3
2	Tổng quan lý thuyết	3
2.1	Kiến trúc của ScaNN	3
2.1.1	Phân vùng (Partitioning)	3
2.1.2	Chấm điểm (Scoring)	4
2.1.3	Sắp xếp lại (Reordering)	4
2.2	So sánh ScaNN với các thuật toán ANN khác	5
2.3	Ứng dụng thực tế của ScaNN	6
2.4	Google Search	6
2.5	Hệ thống gợi ý (Recommendation Systems)	7
2.6	Tích hợp trong TensorFlow	7
3	Phương pháp tiếp cận	8
3.1	Sơ đồ tóm tắt các bước tiếp cận	8
3.2	Thu thập dữ liệu (văn bản)	8
3.3	Sinh vector nhúng (Embedding Generation)	8
3.4	Xây dựng chỉ mục ScaNN và truy vấn top-K.	9
3.5	Brute-force Top-K search	10
3.6	Hiện thực hóa thuật toán SimpleProductQuantization	11
3.6.1	Khởi tạo và Huấn luyện (Training)	11
3.6.2	Mã hóa Vector (Quantization)	11
3.6.3	Truy vấn khoảng cách bất đối xứng (ADC Search)	12
3.6.4	Cơ chế Tinh chỉnh kết quả (Reordering)	12
4	Kết quả thực nghiệm	13
4.1	Quy trình và Môi trường Thực nghiệm	13
4.2	Kết quả so sánh hiệu năng	14
4.2.1	So sánh hiệu năng giữa Brute-force và ScaNN theo các giá trị K khác nhau.	14
4.2.2	Đồ thị trực quan so sánh giữa Brute-force - ScaNN - Manual PQ	14
4.3	Xử lý với tập dữ liệu lớn	14
5	Mở rộng: Kiến trúc GraphRAG - Ứng dụng ScaNN	16
5.1	Xử lý Tiền kỳ: Giải quyết Đồng tham chiếu (Coreference Resolution)	16
5.2	Ứng dụng ScaNN trong Khai phá Tri thức (Link Prediction)	16
5.3	Ứng dụng ScaNN trong Chỉ mục Lai (Hybrid Indexing)	17
6	Giải pháp Mở rộng: Tối ưu hóa 6 Vấn đề cốt lõi của GraphRAG bằng ScaNN	17
6.1	Vấn đề 1: Phân đoạn văn bản chiến thuật (Strategic Chunking)	17
6.2	Vấn đề 2: Hợp nhất thực thể trùng lặp (Entity Disambiguation)	19
6.3	Vấn đề 3: Làm giàu mối quan hệ (Relationship Enrichment)	20
6.4	Vấn đề 4: Tìm kiếm nút mờ (Fuzzy Node Lookup)	21
6.5	Vấn đề 5: Truy vết nguồn gốc (Reference Tracking)	22
6.6	Vấn đề 6: Bộ nhớ ngắn hạn (Short-term Memory)	22
7	Đánh giá	24
7.1	Các tham số chính trong cấu hình ScaNN	24



8	Kết luận	25
8.1	Kết quả đạt được	25
8.2	Hạn chế	25
8.3	Hướng phát triển và Đánh giá phương pháp mở rộng	25
	Tài liệu tham khảo	27

1 Mục tiêu

Đề tài hướng đến việc khám phá và hiện thực thuật toán ScaNN (Scalable Nearest Neighbors)—một giải pháp tìm kiếm vector gần đúng (Approximate Nearest Neighbor Search) hiệu quả và có khả năng mở rộng, được phát triển bởi Google.

Thuật toán ScaNN đã được ứng dụng trong Google Search và TensorFlow, kết hợp ba kỹ thuật cốt lõi: phân vùng (partitioning), lượng hóa vector (quantization) và mã hóa bất đối xứng (asymmetric hashing)[1].

Thông qua quá trình nghiên cứu và cài đặt, đề tài giúp sinh viên tiếp cận, hiểu và ứng dụng một trong những giải pháp hiện đại, tối ưu hiệu suất cao cho bài toán truy xuất vector quy mô lớn.

2 Tổng quan lý thuyết

2.1 Kiến trúc của ScaNN

2.1.1 Phân vùng (Partitioning)

Mục đích và nguyên lý cốt lõi

- Mục tiêu chính của giai đoạn phân vùng là thu hẹp không gian tìm kiếm từ toàn bộ tập dữ liệu xuống một tập con nhỏ hơn, có liên quan. Điều này giúp chuyển đổi một quá trình tìm kiếm tuyến tính, tiêu tốn tài nguyên ($O(N)$), thành một quá trình tìm kiếm dưới tuyến tính hiệu quả hơn nhiều. Về bản chất, phân vùng là một lựa chọn kiến trúc cơ bản, giúp phân rã một vấn đề tìm kiếm lân cận gần đúng quy mô lớn thành nhiều vấn đề nhỏ hơn, cho phép xử lý phân tán trên nhiều máy móc.

Cơ chế: Vai trò của các thuật toán gom cụm

- Để thực hiện việc phân vùng, ScaNN sử dụng các thuật toán gom cụm, chủ yếu là k -means. Thuật toán này hoạt động bằng cách chia một bộ dữ liệu gồm n đối tượng thành k cụm, trong đó mỗi đối tượng được gán vào cụm có tâm (*centroid*) gần nhất. Sự đơn giản và tính trực quan của k -means khiến nó trở thành lựa chọn phù hợp cho việc phân vùng, vì cho phép kiểm soát trực tiếp số lượng phân vùng thông qua tham số k .

Tham số điều chỉnh quan trọng `num_leaves`

- Tham số này tương ứng với số cụm (k) trong giai đoạn phân vùng và quyết định mức độ chi tiết của việc chia nhỏ không gian tìm kiếm. Giá trị của tham số tạo ra sự đánh đổi giữa *thời gian tìm kiếm* và *độ chính xác*:
 - Giá trị cao:** tạo ra nhiều cụm nhỏ hơn, giúp việc lọc ban đầu chính xác hơn. Tuy nhiên, điều này có thể làm tăng thời gian xây dựng chỉ mục và yêu cầu tìm kiếm qua nhiều cụm hơn, ảnh hưởng đến tốc độ truy vấn.
 - Giá trị thấp:** tạo ra ít cụm lớn hơn, giúp tăng tốc độ tìm kiếm ban đầu nhưng có nguy cơ làm giảm độ chính xác tổng thể, do các vector liên quan có thể bị nhóm vào các cụm kém tối ưu.

Trong thực tế, các khuyến nghị cho thấy nên đặt `num_leaves` ở mức khoảng `ROWS/100` để đạt được chất lượng tìm kiếm tối ưu. Khả năng phân phối các phân vùng này là yếu tố cho phép ScaNN xử lý các bộ dữ liệu khổng lồ vượt xa khả năng của một máy tính đơn lẻ.

2.1.2 Chấm điểm (Scoring)

Vấn đề MIPS và sự khác biệt của nó

- Các phương pháp đo lường độ tương đồng của vector bao gồm khoảng cách Euclidean (L_2), độ tương đồng Cosine, và tích vô hướng (*dot product*). ScaNN được thiết kế đặc biệt để giải quyết vấn đề Tìm kiếm Tích vô hướng lớn nhất (Maximum Inner Product Search - MIPS). MIPS khác biệt và phức tạp hơn so với tìm kiếm lân cận gần nhất (NNS) truyền thống khi độ dài (*norm*) của các vector thay đổi.

Hạn chế của lượng tử hóa truyền thống

- Để tăng tốc độ tính toán, các hệ thống tìm kiếm vector thường sử dụng các phương pháp lượng tử hóa để nén các vector chiều cao thành các biểu diễn nhỏ hơn. Các phương pháp lượng tử hóa truyền thống, như *Product Quantization* (PQ)[1], tập trung vào việc giảm thiểu sai số tái tạo (khoảng cách giữa vector gốc và vector đã nén). Tuy nhiên, đây không phải là mục tiêu tối ưu cho MIPS, vì nó có thể dẫn đến việc xếp hạng sai các ứng viên liên quan đến một truy vấn cụ thể.

Đổi mới của ScaNN: Lượng tử hóa vector dị hướng (Anisotropic Vector Quantization - AVQ)[1]

- Điểm đổi mới cốt lõi của ScaNN nằm ở việc sử dụng *Anisotropic Vector Quantization* (AVQ)[1]. AVQ không tập trung vào việc giảm thiểu sai số tái tạo tổng thể mà thay vào đó sử dụng một hàm mất mát “nhận biết điểm số” (*score-aware loss*). Cơ chế này áp dụng hình phạt lớn hơn cho sai số lượng tử hóa xảy ra *song song* với vector gốc.
- Lý do chiến lược đằng sau việc này là sai số ở thành phần song song gây ảnh hưởng bất lợi lớn hơn đến các tích vô hướng cao, vốn là những giá trị mà MIPS đang cố gắng ước tính chính xác. AVQ chấp nhận tăng sai số lượng tử hóa tổng thể để đảm bảo độ chính xác vượt trội cho các tích vô hướng cao. Bằng cách điều chỉnh mục tiêu lượng tử hóa để phù hợp trực tiếp với mục tiêu của MIPS, ScaNN đạt được hiệu suất vượt trội so với các phương pháp dựa trên lượng tử hóa truyền thống, t trội so với các phương pháp lượng tử hóa truyền thống.

2.1.3 Sắp xếp lại (Reordering)

Mục đích của việc sắp xếp lại

- Giai đoạn sắp xếp lại là bước tinh chỉnh cuối cùng trong quá trình tìm kiếm của ScaNN. Sau khi tìm kiếm gần đúng ban đầu (phân vùng và chấm điểm) xác định một tập hợp các ứng viên tiềm năng hàng đầu, giai đoạn sắp xếp lại được sử dụng để tinh chỉnh kết quả nhằm đạt được độ chính xác tối đa.

Cơ chế: Từ gần đúng đến chính xác

- ScaNN sử dụng một cách tiếp cận lại trong giai đoạn này. Nó lấy một tập hợp ứng viên lớn hơn so với số lượng kết quả TopK mà người dùng yêu cầu, sau đó áp dụng tính toán độ tương đồng vết cận chính xác (ví dụ: tích tích vô hướng đầy đủ trên các vector gốc, chưa nén) cho các ứng viên này.
- Cách tiếp cận này kết hợp tốc độ của việc lọc gần đúng ban đầu với độ chính xác cao của tính toán cuối cùng, đảm bảo kết quả đầu ra có độ tin cậy cao.

Các tham số điều chỉnh quan trọng

- Hai tham số quan trọng cho phép người dùng kiểm soát giai đoạn sắp xếp lại:
 - **reorder_k**:
 - * **Định nghĩa**: Xác định số lượng ứng viên hàng đầu từ giai đoạn tìm kiếm ban đầu sẽ được đánh giá lại trong bước tinh chỉnh cuối cùng.
 - * **Đánh đổi**: Giá trị **reorder_k** cao hơn sẽ cải thiện độ chính xác vì nhiều ứng viên được xem xét hơn, nhưng cũng làm tăng độ trễ tìm kiếm. Các khuyến nghị thực tế cho thấy nên đặt **reorder_k** ở mức 2–5 lần số lượng kết quả TopK mong muốn.
 - **with_raw_data**:
 - * **Định nghĩa**: Kiểm soát việc lưu trữ dữ liệu vector gốc (chưa nén) cùng với các biểu diễn đã được lượng tử hóa.
 - * **Yêu cầu**: Giai đoạn sắp xếp lại yêu cầu quyền truy cập vào dữ liệu gốc để thực hiện các phép tính chính xác. Do đó, việc bật tham số này làm tăng yêu cầu bộ nhớ, nhưng đảm bảo kết quả có độ chính xác cao nhất.

Giai đoạn sắp xếp lại đại diện cho bản chất lai của ScaNN, kết hợp một bộ lọc gần đúng nhanh chóng (Phân vùng & Chấm điểm) với một bước kiểm tra vét cạn chính xác và tập trung, giúp cân bằng giữa **tốc độ** và **độ chính xác**.

2.2 So sánh ScaNN với các thuật toán ANN khác

Trong lĩnh vực tìm kiếm hàng xóm gần nhất xấp xỉ (*Approximate Nearest Neighbor – ANN*), nhiều thuật toán đã được phát triển nhằm tối ưu **độ chính xác**, **tốc độ**, **bộ nhớ** và **khả năng mở rộng**. [2] Bảng dưới đây tóm tắt so sánh giữa **ScaNN (Google)** với các phương pháp phổ biến khác: *HNSW*, *FAISS*, *Annoy* và *Brute Force*. [3].

Thuật toán	Nguyên lý	Độ chính xác	Tốc độ	Bộ nhớ	Khả năng mở rộng
Brute Force	Tính khoảng cách với mọi điểm	100%	Rất chậm ($O(n)$)	Trung bình	Không khả thi cho dữ liệu lớn
Annoy	Random projection trees, nhiều cây	Trung bình – Cao	Nhanh (dataset vừa)	Thấp	Vừa
FAISS	Phân cụm IVF, lượng tử hóa PQ, hỗ trợ GPU	Cao – Rất cao	Rất nhanh (GPU)	TB – Cao	Rất lớn
HNSW	Đồ thị nhiều tầng, tìm kiếm qua graph	Rất cao (99%)	Cực nhanh	Cao (RAM lớn)	Lớn
ScaNN	Phân vùng (tree) + lượng tử hóa + re-ranking	Rất cao	Cực nhanh	Trung bình	Rất lớn (tỷ vector)

Table 1: So sánh giữa ScaNN và các thuật toán tìm kiếm hàng xóm gần nhất xấp xỉ (ANN).

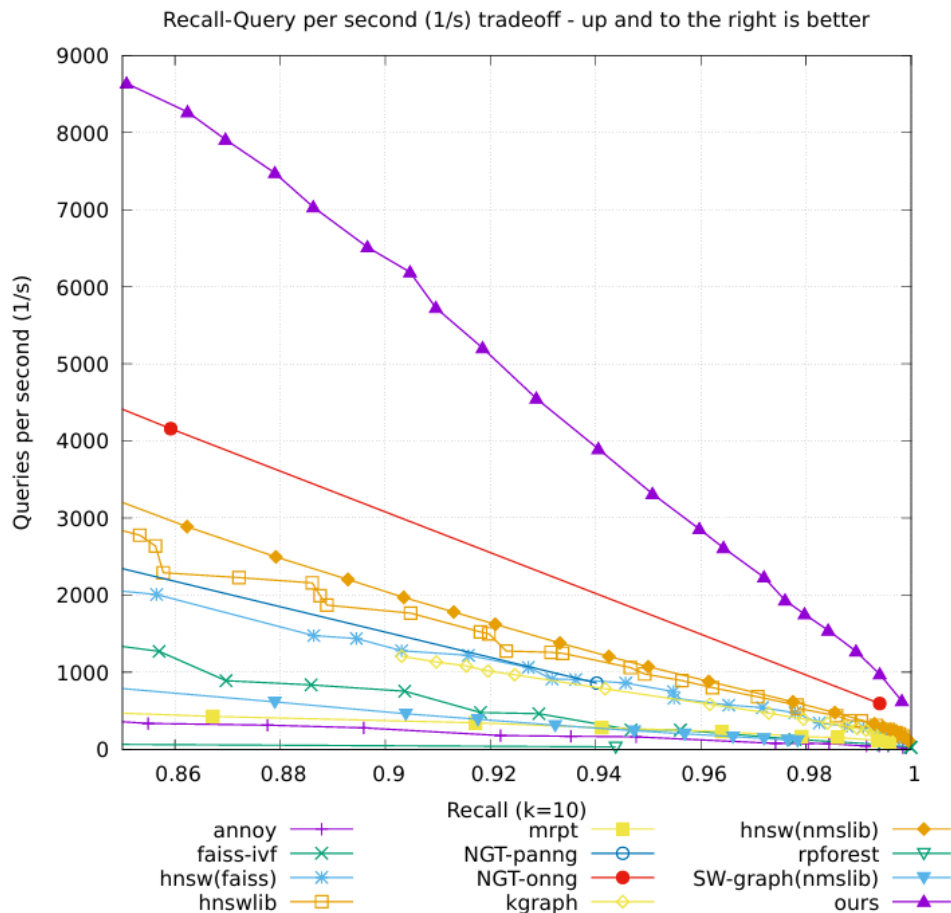


Figure 1: So sánh hiệu năng giữa các thuật toán ANN trên bộ dữ liệu Glove1.2M (Speed-Recall trade-off).

Tóm lại: Brute Force chỉ phù hợp cho tập dữ liệu nhỏ; Annoy [4] thích hợp cho ứng dụng vừa; FAISS và HNSW phổ biến cho hệ thống lớn với độ chính xác cao; trong khi ScaNN được tối ưu cho quy mô rất lớn và các embedding sinh ra từ mô hình học máy.

2.3 Ứng dụng thực tế của ScaNN

2.4 Google Search

- Với quy mô hàng triệu đến hàng tỉ vector, tìm kiếm tuyến tính (brute-force) không còn khả thi trong môi trường sản xuất do độ trễ cao; ScaNN cho phép hệ thống rút ngắn thời gian truy vấn bằng cách chỉ xét các ứng viên tiềm năng. Nhờ đó, Google Search có thể trả kết quả tương tự (văn bản/hình ảnh) trong tích tắc, đáp ứng yêu cầu thời gian thực của người dùng [8]
- Trong ứng dụng thực tế như tìm kiếm hình ảnh hay văn bản ngữ nghĩa, ScaNN cho phép xác định nhanh các kết quả tương đồng bằng cách so sánh với các vector gần nhất thay vì

toàn bộ tập dữ liệu. Nhờ đó, Google có thể trả kết quả tức thời cho hàng tỉ truy vấn mỗi ngày.

- Kết quả thực nghiệm cho thấy ScaNN đạt tốc độ vượt trội trong vùng recall cao, khiến nó đặc biệt phù hợp cho các tác vụ tìm kiếm văn bản và hình ảnh quy mô web.

2.5 Hệ thống gợi ý (Recommendation Systems)

Trong các hệ thống gợi ý quy mô lớn, nhiệm vụ cốt lõi là tìm ra các mục liên quan hoặc giống với sở thích của người dùng. ScaNN thường được triển khai ở giai đoạn *truy vấn ứng viên (retrieval stage)*, nơi cần tìm nhanh tập hợp k ứng viên gần nhất từ kho dữ liệu chứa hàng triệu đến hàng tỉ vector.

- **TensorFlow Recommenders (TFR)**[7]: minh họa cách dùng ScaNN để gợi ý phim từ tập dữ liệu MovieLens. Thay vì so sánh tất cả các vector phim, ScaNN chỉ xét một phần embedding liên quan, giúp thời gian truy vấn cải thiện đáng kể. ScaNN trả về danh sách *top-k* phim gần nhất trong embedding space chỉ trong vài mili-giây.
- **YouTube**[9]: sử dụng ANN (bao gồm ScaNN) để chọn ra vài trăm video phù hợp nhất từ hàng tỉ video trước khi đưa vào giai đoạn xếp hạng (ranking).
- **E-commerce (Amazon, Shopee, Lazada)**[10]: tìm nhanh các sản phẩm tương tự hoặc sản phẩm người dùng có khả năng mua, ví dụ: “người xem áo sơ mi này cũng thường mua quần tây này.”
- **Music & Movie recommendation (Spotify, Netflix)**: xác định bài hát hoặc phim gần nhất với sở thích người dùng dựa trên embedding học được từ lịch sử nghe/xem.

2.6 Tích hợp trong TensorFlow

ScaNN được tích hợp trong **TensorFlow Recommenders (TFR)**[6][7] như một lựa chọn chỉ mục cho bài toán truy vấn embedding quy mô lớn. Việc này cho phép thực hiện toàn bộ quy trình từ huấn luyện mô hình nhúng, xây dựng chỉ mục đến truy vấn top-K ngay trong cùng một framework, giúp giảm chi phí tích hợp và thời gian triển khai.

Quy trình điển hình:

- Huấn luyện hai mạng nhúng (encoder) riêng cho truy vấn và ứng viên.
- Xuất embedding ứng viên và tạo chỉ mục ScaNN.
- Truy vấn top-K ứng viên, sau đó chuyển cho mô-đun ranking để đánh giá chi tiết.

Cách tiếp cận này giúp tách biệt giai đoạn *retrieval* (tối ưu tốc độ và khả năng mở rộng) và *ranking* (tối ưu độ chính xác). ScaNN hỗ trợ tinh chỉnh các tham số như số phân vùng, số leaf được quét và cấp độ lượng tử hóa để cân bằng giữa **tốc độ và độ chính xác**.

3 Phương pháp tiếp cận

3.1 Sơ đồ tóm tắt các bước tiếp cận

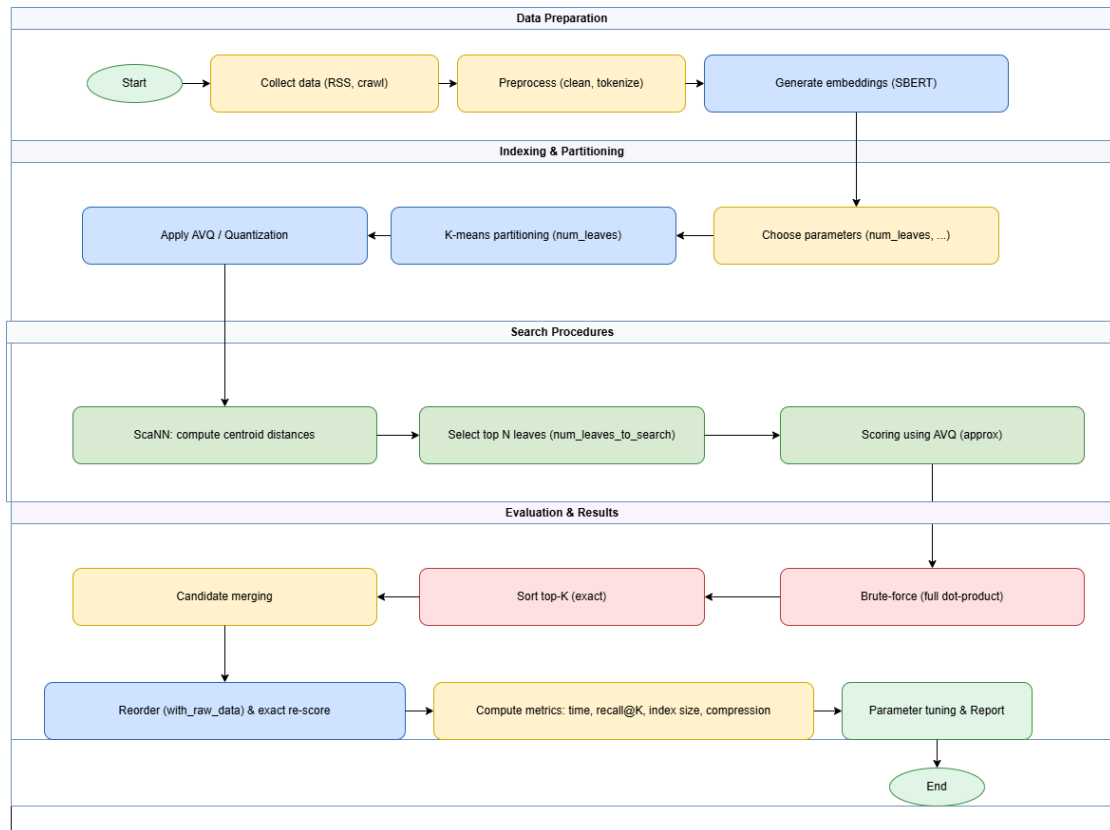


Figure 2: Sơ đồ tóm tắt các bước tiếp cận

3.2 Thu thập dữ liệu (văn bản)

Các thao tác chính:

- **Khai báo nguồn dữ liệu:** Tập hợp danh sách các đường dẫn RSS từ nhiều trang báo lớn như VnExpress, Tuổi Trẻ, Thanh Niên, ZingNews, và Dân Trí.
- **Thu thập tin tức:** Duyệt qua từng nguồn RSS, lấy tiêu đề và đường dẫn của các bài viết, sau đó lưu vào danh sách để dùng sau này.

3.3 Sinh vector nhúng (Embedding Generation)

```

from sentence_transformers import SentenceTransformer
import numpy as np
model = SentenceTransformer("keepitreal/vietnamese-sbert")
embeddings = model.encode(texts, batch_size=32, show_progress_bar=True)
  
```

Các thao tác chính:

- **Khởi tạo mô hình:** Sử dụng mô hình Vietnamese-SBERT từ thư viện `SentenceTransformer`, được huấn luyện đặc biệt cho tiếng Việt, giúp biểu diễn câu hoặc đoạn văn thành vector ngữ nghĩa.
- **Sinh embedding:** Truyền danh sách các văn bản (`texts`) đã thu thập vào mô hình để sinh ra các vector nhúng (`embeddings`), trong đó mỗi bài viết được biểu diễn bằng một vector số học có kích thước cố định (768 chiều).
- **Lưu trữ và sử dụng:** Các vector embedding này là đầu vào cho các bước tiếp theo như tính độ tương đồng, tìm kiếm tin tức liên quan hoặc gom cụm bài viết có nội dung gần nhau.

3.4 Xây dựng chỉ mục ScaNN và truy vấn top-K.

```
import scann
import numpy as np
searcher = scann.scann_ops_pybind.builder(
    embeddings,
    10,
    "dot_product"
).tree(
    num_leaves=32,
    num_leaves_to_search=16,
    training_sample_size=5000
).score_ah(
    2,
    anisotropic_quantization_threshold=0.25
).reorder(100).build()

# Truy vấn top-K
query = "Việt Nam"
q_emb = model.encode(query).astype(np.float32)
neighbors, distances = searcher.search(q_emb)

for i, n in enumerate(neighbors):
    print(f"{i+1}. {texts[n]} (score={distances[i]:.4f})")
```

Các thao tác chính:

- **Khởi tạo và xây dựng chỉ mục ScaNN:** Sử dụng thư viện `scann` (Scalable Nearest Neighbors) để xây dựng cấu trúc chỉ mục tìm kiếm hiệu năng cao cho tập dữ liệu vector nhúng (`embeddings`). Quy trình bao gồm ba giai đoạn chính: Phân vùng (Partitioning), Lượng tử hóa (Quantization)[1] và Tinh chỉnh (Re-ranking).
 - **Giai đoạn khởi tạo:** Hàm `builder()` được gọi với ba tham số cơ bản:
 - * `embeddings`: Ma trận vector đầu vào cần lập chỉ mục.
 - * `10`: Số lượng láng giềng gần nhất (k -NN) mặc định cần truy xuất.
 - * `"dot_product"`: Độ đo khoảng cách sử dụng tích vô hướng.
 - **Giai đoạn phân vùng không gian (Tree):** Hàm `tree()` cấu hình việc chia không gian dữ liệu để giảm không gian tìm kiếm:

- * `num_leaves=32`: Chia toàn bộ tập dữ liệu thành 32 phân vùng (cụm).
- * `num_leaves_to_search=16`: Khi truy vấn, hệ thống sẽ chỉ duyệt qua 16 phân vùng có xác suất chứa kết quả cao nhất (xấp xỉ 50% không gian dữ liệu).
- * `training_sample_size=5000`: Sử dụng một tập mẫu ngẫu nhiên gồm 5000 vector để huấn luyện thuật toán phân cụm (K-Means), giúp giảm thời gian xây dựng chỉ mục.
- **Giai đoạn lượng tử hóa và chấm điểm (Score AH)**: Hàm `score_ah()` áp dụng kỹ thuật *Asymmetric Hashing* với cải tiến lượng tử hóa dị hướng:
 - * Tham số 2 (Dimensions per block): Nén dữ liệu bằng cách gộp mỗi 2 chiều của vector gốc thành một khối để tối ưu hóa tính toán trên phần cứng.
 - * `anisotropic_quantization_threshold=0.25`: Hệ số ngưỡng cho lượng tử hóa dị hướng. Giá trị này giúp mô hình ưu tiên bảo toàn độ chính xác của tích vô hướng bằng cách giảm thiểu sai số song song với vector gốc, khắc phục nhược điểm của lượng tử hóa truyền thống.
- **Giai đoạn tinh chỉnh (Reorder)**: Hàm `reorder(100)` thực hiện bước xếp hạng lại (re-ranking). Hệ thống sẽ lấy ra 100 ứng viên tốt nhất từ bước tìm kiếm xấp xỉ, sau đó tính toán khoảng cách chính xác trên dữ liệu gốc để lọc ra 10 kết quả tối ưu nhất trả về cho người dùng.
- **Mã hoá truy vấn**: Biến `query` chứa câu truy vấn được đưa vào mô hình SBERT để sinh ra vector nhúng tương ứng `q_emb`.
- **Thực hiện tìm kiếm top-K**: Gọi hàm `search()` trên đối tượng `searcher` để tìm ra các bài viết có độ tương đồng cao nhất với truy vấn. Kết quả trả về gồm:
 - `neighbors`: danh sách chỉ số các bài viết gần nhất trong `texts`,
 - `distances`: điểm tương đồng (score) tương ứng cho từng bài.
- **Hiển thị kết quả**:

3.5 Brute-force Top-K search

```
import numpy as np
# vector query
query = "Việt Nam"
q_emb = model.encode(query).astype(np.float32)
# brute-force dot product
scores = embeddings @ q_emb
K = 50
top_k_idx = np.argsort(-scores)[:K]
top_k_scores = scores[top_k_idx]
for i, idx in enumerate(top_k_idx):
    print(f"{i+1}. {texts[idx]} (score={top_k_scores[i]:.4f})")
```

Các thao tác chính:

- **Mã hoá truy vấn**
- **Tính điểm tương đồng**: Dùng phép nhân ma trận (`@`) giữa toàn bộ tập vector nhúng `embeddings` và vector truy vấn `q_emb` để tính độ tương đồng theo *dot product*. Kết quả là một mảng `scores`, trong đó mỗi phần tử biểu thị mức độ liên quan giữa bài viết và truy vấn.

- **Xác định top-K kết quả:** Sắp xếp chỉ số của các bài viết theo điểm tương đồng giảm dần bằng `np.argsort(-scores)`, sau đó lấy $K = 50$ bài có điểm cao nhất.
- **Hiển thị kết quả**

3.6 Hiện thực hóa thuật toán SimpleProductQuantization

Để minh họa cơ chế hoạt động lõi của phương pháp nén vector, nhóm xây dựng lớp `SimpleProductQuantization` bao gồm các giai đoạn: Huấn luyện (Training), Mã hóa (Encoding), Truy vấn xấp xỉ (ADC Search) và Tinh chỉnh kết quả (Reordering).

3.6.1 Khởi tạo và Huấn luyện (Training)

Quá trình huấn luyện chia không gian vector D -chiều ban đầu thành M không gian con (subspaces). Trên mỗi không gian con, thuật toán K-Means được áp dụng để tìm ra k tâm cụm (centroids), tạo thành các tập mã (codebooks).

```
class SimpleProductQuantization:
    def __init__(self, M=8, k_centroids=256):
        self.M = M # Số lượng đoạn con (subspaces)
        self.k = k_centroids # Số tâm cụm (thường là 256 để khớp 8-bit)
        self.codebooks = []
        self.d_sub = None

    def fit(self, X):
        N, D = X.shape
        self.d_sub = D // self.M
        # Duyệt qua từng đoạn con để huấn luyện K-Means độc lập
        for m in range(self.M):
            start = m * self.d_sub
            end = (m + 1) * self.d_sub
            X_sub = X[:, start:end]

            # K-Means clustering cho đoạn m
            kmeans = KMeans(n_clusters=self.k, n_init=10, random_state=42)
            kmeans.fit(X_sub)
            self.codebooks.append(kmeans.cluster_centers_)
```

3.6.2 Mã hóa Vector (Quantization)

Giai đoạn mã hóa thay thế mỗi đoạn vector con bằng chỉ số (index) của tâm cụm gần nhất trong codebook tương ứng. Quá trình này giảm kích thước dữ liệu từ các số thực (float32) xuống các số nguyên (uint8).

```
def encode(self, X):
    N = X.shape[0]
    X_encoded = np.zeros((N, self.M), dtype=np.uint8)

    for m in range(self.M):
        # Trích xuất dữ liệu đoạn m
        start = m * self.d_sub
        end = (m + 1) * self.d_sub
        X_sub = X[:, start:end]
```

```
centroids = self.codebooks[m]

# Tính khoảng cách và tìm centroid gần nhất
dists = np.linalg.norm(X_sub[:, None] - centroids[None, :], axis=2)
X_encoded[:, m] = np.argmin(dists, axis=1)

return X_encoded
```

3.6.3 Truy vấn khoảng cách bất đối xứng (ADC Search)

Để tăng tốc độ truy vấn, thay vì giải nén vector, ta sử dụng kỹ thuật tính toán khoảng cách bất đối xứng (Asymmetric Distance Computation). Hệ thống tính trước bảng khoảng cách (Lookup Table) giữa vector truy vấn và tất cả các tâm cụm, sau đó cộng dồn khoảng cách dựa trên các chỉ số đã lưu.

```
def search(self, query, X_encoded, top_k=10):
    # 1. Precompute: Tạo bảng tra khoảng cách (Distance Table)
    distance_table = np.zeros((self.M, self.k))
    for m in range(self.M):
        start = m * self.d_sub
        end = (m + 1) * self.d_sub
        q_sub = query[start:end]
        # Khoảng cách từ query sub-vector đến tất cả centroids
        distance_table[m, :] = np.linalg.norm(self.codebooks[m] - q_sub, axis=1)

    # 2. Lookup & Sum: Cộng dồn khoảng cách trực tiếp trên mã nén
    scores = np.zeros(X_encoded.shape[0])
    for m in range(self.M):
        col_indices = X_encoded[:, m]
        scores += distance_table[m, col_indices] # Fancy indexing

    # 3. Ranking: Lấy Top-K có khoảng cách nhỏ nhất
    top_indices = np.argsort(scores)[:top_k]
    return top_indices, scores[top_indices]
```

3.6.4 Cơ chế Tinh chỉnh kết quả (Reordering)

Do quá trình lượng tử hóa gây ra sai số, bước Reordering được thực hiện bằng cách lấy ra tập ứng viên lớn hơn ($K \times factor$) từ bước tìm kiếm thô, sau đó tính toán lại khoảng cách chính xác trên dữ liệu gốc để đảm bảo độ chính xác (Recall).

```
def search_with_reordering(pq_model, query_vec, X_encoded, X_raw, top_k=10, factor=10):
    # Giai đoạn 1: Tìm kiếm thô (Coarse Search) trên dữ liệu nén
    candidates_k = top_k * factor
    candidate_indices, _ = pq_model.search(query_vec, X_encoded, top_k=candidates_k)

    # Giai đoạn 2: Tìm kiếm tinh (Fine Search) trên dữ liệu gốc
    candidate_vectors = X_raw[candidate_indices]

    # Tính tích vô hướng chính xác (Exact Dot Product)
    exact_scores = candidate_vectors @ query_vec

    # Sắp xếp lại cục bộ để lấy top_k tốt nhất
```

```
best_local_indices = np.argsort(-exact_scores)[:top_k]
final_indices = candidate_indices[best_local_indices]

return final_indices, exact_scores[best_local_indices]
```

4 Kết quả thực nghiệm

4.1 Quy trình và Môi trường Thực nghiệm

Để đánh giá toàn diện hiệu năng của giải pháp, nhóm thiết lập quy trình thực nghiệm bao gồm ba giai đoạn chính: đánh giá chất lượng lượng tử hóa, so sánh hiệu năng truy vấn và phân tích hiệu quả lưu trữ.

1. Đánh giá sai số tái tạo (Reconstruction Error Analysis):

Trước khi thực hiện truy vấn, chất lượng của quá trình nén vector (Product Quantization) được kiểm định thông qua chỉ số Sai số Trung bình Bình phương (MSE). Vector xấp xỉ $\hat{x}$ được tái tạo từ mã nén (codebooks) và so sánh với vector gốc x theo công thức:

$$MSE = \frac{1}{N} \sum_{i=1}^N ||x_i - \hat{x}_i||^2$$

Giá trị MSE càng nhỏ thể hiện vector nén càng giữ được đặc trưng của vector gốc, tiền đề cho độ chính xác (Recall) cao.

2. Thiết lập kịch bản kiểm thử (Benchmark Protocol):

Thực nghiệm so sánh ba phương pháp: *Brute-Force* (duyệt toàn bộ - làm cơ sở chân lý), *ScaNN* (thư viện tối ưu) và *Manual PQ* (cài đặt thủ công).

- **Tập truy vấn (Query Set):** Lấy mẫu ngẫu nhiên 100 vector ($N_{query} = 100$) từ tập dữ liệu để đảm bảo tính khách quan.
- **Ground Truth:** Kết quả trả về từ thuật toán Brute-Force (tính khoảng cách chính xác với toàn bộ dữ liệu) được coi là đáp án chuẩn (Recall = 100%).

3. Các chỉ số đánh giá hiệu năng (Evaluation Metrics):

Hệ thống được đo lường dựa trên ba tiêu chí trọng yếu:

- **Độ chính xác (Recall@K):** Tỷ lệ trùng khớp giữa tập kết quả trả về của mô hình (S_{model}) và tập kết quả chuẩn (S_{gt}) tại top K láng giềng (với $K = 10$):

$$Recall@K = \frac{|S_{model} \cap S_{gt}|}{K} \times 100\%$$

- **Độ trễ (Latency):** Thời gian trung bình để xử lý một truy vấn, tính bằng mili-giây (ms), đo lường từ lúc nhận vector đến khi trả về danh sách kết quả.
- **Hiệu quả lưu trữ (Storage Efficiency):** So sánh dung lượng của chỉ mục ($Size_{index}$) so với dữ liệu gốc ($Size_{raw}$). Tỷ lệ nén được xác định bởi:

$$Ratio = \frac{Size_{raw}}{Size_{index}}$$

Trong đó, $Size_{index}$ của ScaNN được tính thông qua việc serialize cấu trúc cây và bảng băm ra bộ nhớ ngoài.

4.2 Kết quả so sánh hiệu năng

4.2.1 So sánh hiệu năng giữa Brute-force và ScaNN theo các giá trị K khác nhau.

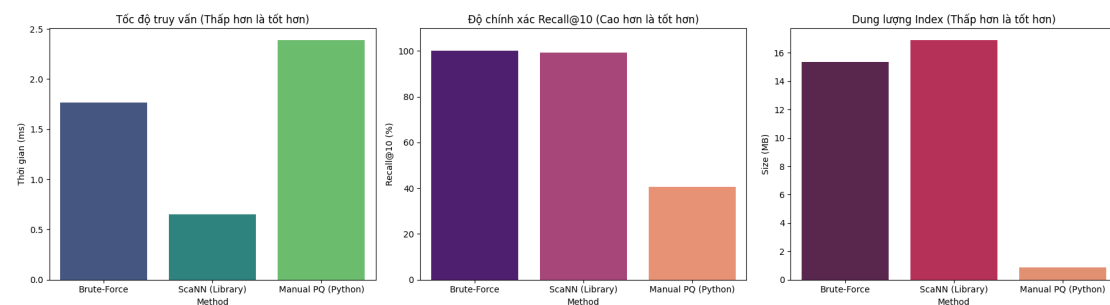
Kết quả được thực hiện với tập dữ liệu gồm 5000 phần tử và 200 truy vấn. Các giá trị K được thay đổi để khảo sát ảnh hưởng đến tốc độ và độ chính xác.

Top-K	Thời gian BF (s)	Thời gian ScaNN (s)	Speed-up	Recall@K	Tỷ lệ nén
10	0.002436	0.000446	5.46x	0.9740	0.86
20	0.00233	0.000429	5.44x	0.9733	0.86
50	0.002279	0.000417	5.46x	0.9659	0.87
100	0.002313	0.00044	5.26x	0.8827	0.87

Kết quả cho thấy:

- ScaNN giúp rút ngắn thời gian truy vấn trung bình **5 lần** so với Brute-force.
- Độ chính xác trung bình (recall@K) duy trì ở mức cao, từ **0.88 đến 0.97**.
- Khi K tăng, recall giảm, speed-up giảm nhẹ do số lượng kết quả cần xử lý lớn hơn.
- Tỷ lệ nén của ScaNN đạt khoảng **0.86–0.87**

4.2.2 Đồ thị trực quan so sánh giữa Brute-force - ScaNN - Manual PQ



4.3 Xử lý với tập dữ liệu lớn

Ở quy mô dữ liệu ban đầu gồm 5000 văn bản (khoảng 15MB), sự khác biệt trong hiệu năng giữa ba mục tiêu *thời gian truy vấn*, *độ chính xác truy vấn (Recall)* và *mức độ nén dữ liệu* vẫn chưa thể hiện rõ ràng. Do đó, để đánh giá toàn diện hơn khả năng mở rộng và hiệu quả tối ưu của ScaNN, nhóm tiến hành thử nghiệm trên tập dữ liệu lớn hơn với:

$$N = 4,000,000, \quad D = 128, \quad \text{kiểu dữ liệu: float32}$$

Tổng dung lượng tương ứng xấp xỉ ~ 2 GB. Mô hình được triển khai dưới dạng phiên bản nén $4\times$ và chạy trên CPU, với các tham số cấu hình:

$$\text{num_leaves} = 2048, \quad \text{n_search} = 800, \quad \text{reorder_k} = 5000, \quad \text{dims_per_block} = 1.$$

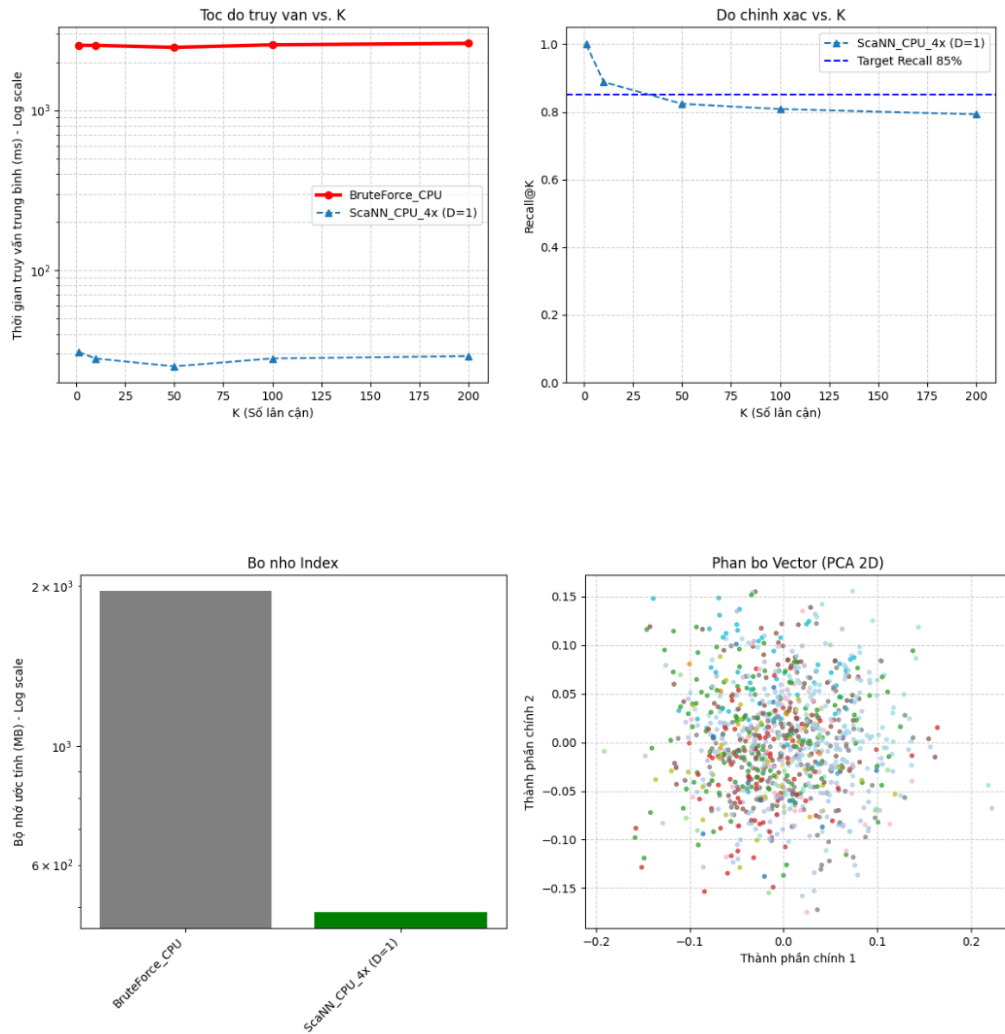
Nhóm thực hiện truy vấn với $n_q = 50$ truy vấn ngẫu nhiên, ghi nhận **thời gian truy vấn trung bình** và **Recall@K** với các giá trị $K = 1, 10, 50, 100, 200$.

Table 2: So sánh chi phí xây dựng và bộ nhớ ($N = 4,000,000$; $D = 128$; CPU)

Phương pháp	Thời gian Build (ms)	Bộ nhớ (MB)
Brute Force (CPU)	254.33	1953.12
ScaNN (CPU, nén 4x, D=1)	304349.85	489.28

Table 3: So sánh hiệu năng truy vấn và Recall@K ($Q = 50$ truy vấn ngẫu nhiên)

K	Time BF (ms)	Time ScaNN (ms)	Recall ScaNN
1	2546.43	30.88	1.000
10	2546.14	28.03	0.888
50	2473.34	25.05	0.8236
100	2566.32	28.09	0.8082
200	2619.57	29.06	0.7928



5 Mở rộng: Kiến trúc GraphRAG - Ứng dụng ScaNN

Để khắc phục các hạn chế của phương pháp tìm kiếm vector truyền thống, nhóm đề xuất và hiện thực hóa kiến trúc **GraphRAG** (Graph Retrieval-Augmented Generation). Hệ thống được nâng cấp với quy trình xử lý ScaNN đa giai đoạn, đóng vai trò vừa là công cụ làm sạch dữ liệu, vừa là bộ máy truy vấn lại.

5.1 Xử lý Tiền kỳ: Giải quyết Đồng tham chiếu (Coreference Resolution)

Các mô hình nhúng thường gặp khó khăn với các đại từ thay thế. Tại giai đoạn khởi tạo, nhóm tích hợp mô hình **fastcoref** để xử lý văn bản đầu vào. *Lưu ý: Mặc dù mô hình được tải lên, logic xử lý chính tập trung vào việc phân đoạn ngữ nghĩa bằng ScaNN để đảm bảo các chunk văn bản có tính độc lập cao nhất.*

5.2 Ứng dụng ScaNN trong Khai phá Tri thức (Link Prediction)

Để khắc phục sự thừa thớt của Đồ thị Tri thức, nhóm sử dụng ScaNN trong **Giai đoạn 3** của pipeline để dự đoán các liên kết ẩn. Thay vì chỉ tìm các node giống hệt nhau, hệ thống tìm kiếm các cặp thực thể nằm trong "vùng ngữ nghĩa liên quan" (Semantic Related Zone). Hệ thống sử dụng 'entity_searcher' đã được huấn luyện để quét toàn bộ các vector thực thể.

```
print(" Giai đoạn 3: ScaNN Link Prediction...")
new_links = []
existing_pairs = set((t['subject'], t['object']) for t in clean_triples)

for i, vec in enumerate(tqdm(entity_vecs, desc="Predicting Links")):
    # Tìm 5 láng giềng gần nhất cho mỗi thực thể
    neighbors, dists = entity_searcher.search(vec, final_num_neighbors=5)

    for idx, dist in zip(neighbors, dists):
        if i == idx: continue

        # Logic lọc ngưỡng: Chỉ lấy vùng tương đồng từ 0.85 đến 0.96
        # (> 0.96 thường là trùng lặp tên, < 0.85 là không liên quan)
        if 0.85 < dist < 0.96:
            e1, e2 = entities[i], entities[idx]

            if (e1, e2) not in existing_pairs and (e2, e1) not in existing_pairs:
                new_links.append({
                    "subject": e1,
                    "relation": "SCANN_SEMANTIC_LINK",
                    "object": e2,
                    "source_chunk_id": "scann_prediction"
                })
            existing_pairs.add((e1, e2))
```

Cơ chế hoạt động thực tế:

1. Hệ thống tái sử dụng 'entity_searcher' (đã xây dựng ở bước làm sạch dữ liệu).

- Thiết lập ngưỡng lọc (Filter Threshold): Chỉ chấp nhận độ tương đồng trong khoảng **0.85 đến 0.96**.
- Các cặp thỏa mãn sẽ được tạo quan hệ mới mang nhãn `SCANN_SEMANTIC_LINK`, giúp đồ thị dày đặc hơn và kết nối được các khái niệm liên quan mà không cần trích xuất trực tiếp từ văn bản.

5.3 Ứng dụng ScaNN trong Chỉ mục Lai (Hybrid Indexing)

Tại **Giai đoạn 4**, ScaNN được sử dụng để xây dựng chỉ mục cuối cùng (Final Index). Điểm đặc biệt của kiến trúc này là sự hợp nhất giữa hai loại dữ liệu: *Text Chunks* (đoạn văn bản) và *Entities* (thực thể đồ thị) vào chung một không gian vector.

```
print(" Giai đoạn 4: Building Final Scann Index...")

# 1. Hợp nhất Corpus: Text Chunks + Cleaned Entities
final_entities = list(set([t['subject'] for t in clean_triples] + \
                          [t['object'] for t in clean_triples]))
corpus = [c['content'] for c in all_chunks] + final_entities

index_map = {}
# 2. Đánh dấu metadata để phân loại khi truy xuất
for i, item in enumerate(corpus):
    if i < len(all_chunks):
        index_map[i] = {"type": "text", "content": item}
    else:
        index_map[i] = {"type": "entity", "content": item}

# 3. Mã hóa toàn bộ và Build Index (sử dụng hàm helper build_scann_index)
all_vectors = embedding_model.encode(corpus, show_progress_bar=True)
searcher = build_scann_index(all_vectors, num_neighbors=20)
```

Ý nghĩa chiến lược: Việc sử dụng hàm 'build_scann_index' (được định nghĩa tùy biến trong giúp tự động tinh chỉnh tham số cấu hình ScaNN (như 'num_leaves', 'anisotropic_quantization_threshold') dựa trên kích thước dữ liệu. Kết quả là một chỉ mục lai cho phép truy vấn trả về đồng thời cả nội dung văn bản gốc và các thực thể đồ thị, phục vụ cho bước suy luận sau này.

6 Giải pháp Mở rộng: Tối ưu hóa 6 Vấn đề cốt lõi của GraphRAG bằng ScaNN

Trong quá trình xây dựng hệ thống GraphRAG, **nhóm** nhận diện 6 thách thức kỹ thuật lớn làm ảnh hưởng đến độ chính xác và hiệu năng. Thay vì sử dụng nhiều công cụ rời rạc, **nhóm** tận dụng thư viện **ScaNN** như một giải pháp đa năng để giải quyết toàn diện các vấn đề này. Dưới đây là chi tiết hiện thực hóa cho từng vấn đề dựa trên mã nguồn thực tế của hệ thống.

6.1 Vấn đề 1: Phân đoạn văn bản chiến thuật (Strategic Chunking)

Các phương pháp cắt văn bản cố định (Fixed-size chunking) thường làm gãy ngữ nghĩa của câu. **Giải pháp:** Nhóm sử dụng ScaNN để so sánh độ tương đồng giữa câu hiện tại và nhóm câu đang xét. Một "chỉ mục ScaNN cục bộ" được dựng lên ngay trong quá trình xử lý văn bản để quyết định điểm cắt dựa trên ngưỡng ngữ nghĩa (threshold).

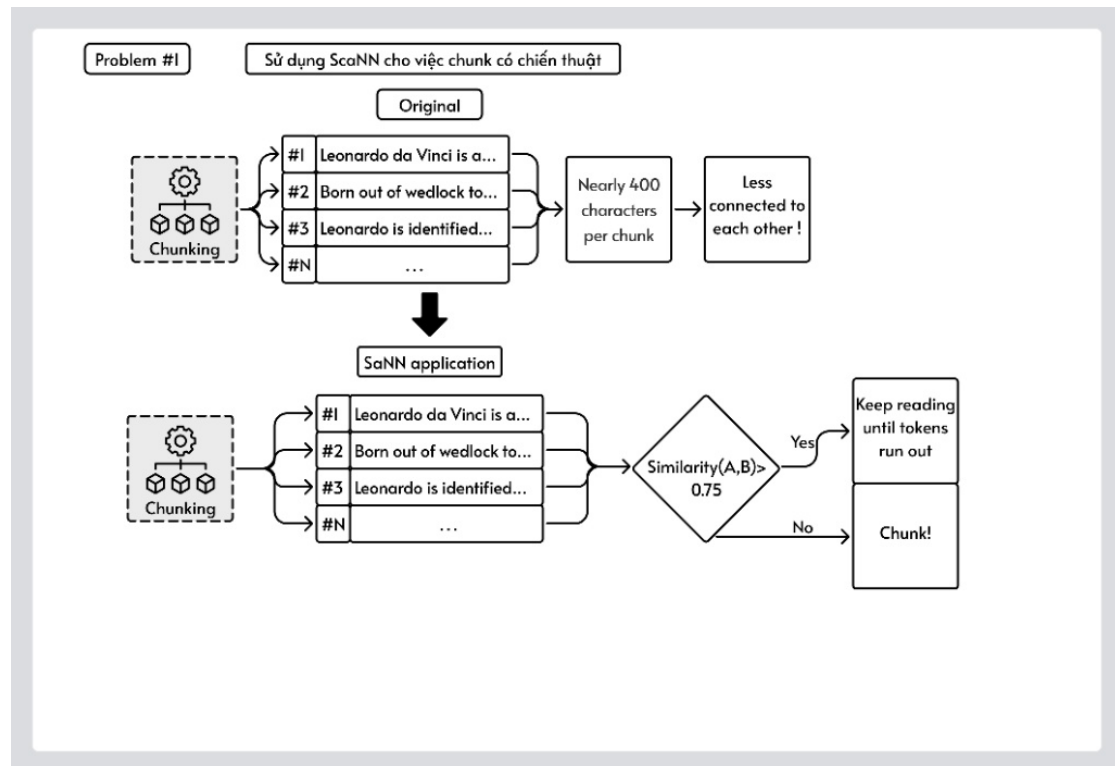


Figure 3: Chiến thuật Semantic Chunking sử dụng ScaNN Indexing

```
# --- 1. CHUNKING LOGIC ---
def scann_semantic_chunking(text, model, threshold=0.5):
    sentences = re.split(r'(?<=[.?!])\s+', text)
    if not sentences: return []
    embeddings = model.encode(sentences, show_progress_bar=False)
    searcher = build_scann_index(embeddings, num_neighbors=5)
    final_chunks = []
    current_group = [sentences[0]]
    current_vec = embeddings[0]

    for i in range(1, len(sentences)):
        sim_score = np.dot(current_vec, embeddings[i]) / (np.linalg.norm(current_vec) *
        ↪ np.linalg.norm(embeddings[i]) + 1e-9)
        if sim_score >= threshold:
            current_group.append(sentences[i])
            current_vec = (current_vec + embeddings[i]) / 2.0
        else:
            final_chunks.append(" ".join(current_group))
            current_group = [sentences[i]]
            current_vec = embeddings[i]

    if current_group: final_chunks.append(" ".join(current_group))
    return final_chunks
```

6.2 Vấn đề 2: Hợp nhất thực thể trùng lặp (Entity Disambiguation)

Một thực thể có thể xuất hiện dưới nhiều biến thể tên gọi, tạo ra các nút thừa. **Giải pháp:** Nhóm sử dụng ScaNN để thực hiện *Entity Resolution*. Hệ thống tính toán vector cho tất cả thực thể, sau đó tìm kiếm lân cận (Neighbors). Nếu khoảng cách tương đồng rất cao (ngưỡng > 0.96), các thực thể sẽ được gộp về một tên chuẩn duy nhất (Canonical Name).

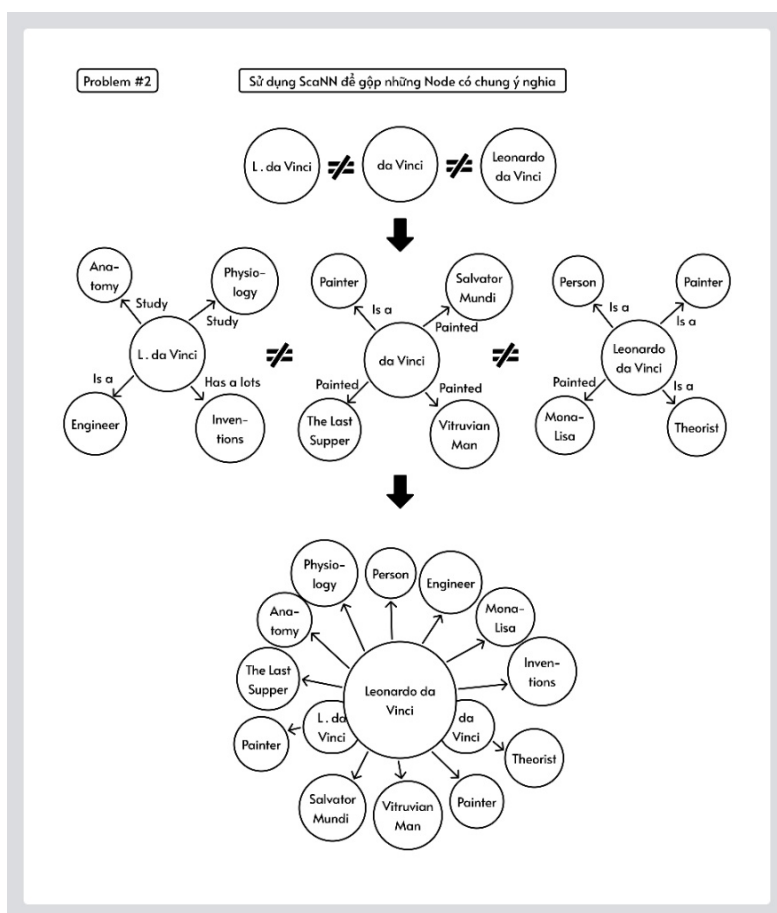


Figure 4: Sử dụng ScaNN Search để làm sạch và gộp thực thể

```
# Giai đoạn 2: ScaNN Cleaning (Entity Resolution)
entity_searcher = build_scann_index(entity_vecs, num_neighbors=10)
canonical_map = {}
for i, ent in enumerate(tqdm(entities)):
    # Tìm kiếm các thực thể có vector gần nhất
    neighbors, dists = entity_searcher.search(entity_vecs[i], final_num_neighbors=10)
    cluster = []
    for idx, dist in zip(neighbors, dists):
        # Ngưỡng 0.96: Rất chắc chắn là cùng một thực thể (VD: "Leonardo" & "Leonardo da
        # Vinci")
        if dist > 0.96:
            cluster.append(idx)
```

```
# Chọn tên dài nhất làm tên chuẩn (Canonical)
if cluster:
    cluster_names = [entities[ix] for ix in cluster]
    canonical = max(cluster_names, key=len)
    for name in cluster_names:
        canonical_map[name] = canonical
```

6.3 Vấn đề 3: Làm giàu mối quan hệ (Relationship Enrichment)

Đồ thị tri thức thường bị thưa thớt (sparse) do giới hạn của mô hình trích xuất. **Giải pháp:** Nhóm tận dụng ScaNN để thực hiện *Link Prediction*. Hệ thống quét tìm các cặp thực thể có độ tương đồng ngữ nghĩa nằm trong khoảng "liên quan nhưng không trùng lặp" ($0.85 < dist < 0.96$) và tự động bổ sung cạnh SCANN_SEMANTIC_LINK.

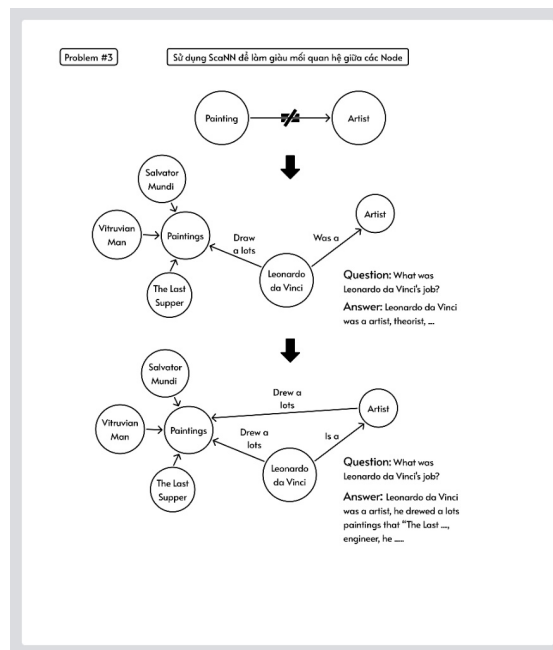


Figure 5: Dự đoán liên kết ẩn (Link Prediction) bằng vector similarity

```
# Giai đoạn 3: ScaNN Link Prediction
new_links = []
for i, vec in enumerate(tqdm(entity_vecs)):
    neighbors, dists = entity_searcher.search(vec, final_num_neighbors=5)
    for idx, dist in zip(neighbors, dists):
        if i == idx: continue
        # Vùng "Semantic Related" (0.85 < dist < 0.96)
        # VD: "Mona Lisa" (Tranh) và "Leonardo" (Họa sĩ) sẽ có vector gần nhau
        if 0.85 < dist < 0.96:
            e1, e2 = entities[i], entities[idx]
            if (e1, e2) not in existing_pairs:
                new_links.append({
                    "subject": e1,
```

```
"relation": "SCANN_SEMANTIC_LINK",
"object": e2
})
```

6.4 Vấn đề 4: Tìm kiếm nút mờ (Fuzzy Node Lookup)

Người dùng thường nhập tên không chính xác hoặc chỉ hỏi chung chung. Việc khớp chuỗi (string matching) truyền thống thường thất bại. **Giải pháp:** Nhóm xây dựng một **Hybrid ScaNN Index** chứa cả Text Chunks và Entities. Khi truy vấn, vector của câu hỏi sẽ quét qua index này để tìm ra các thực thể liên quan nhất (Fuzzy Match) làm điểm neo (Anchor) cho việc mở rộng đồ thị.

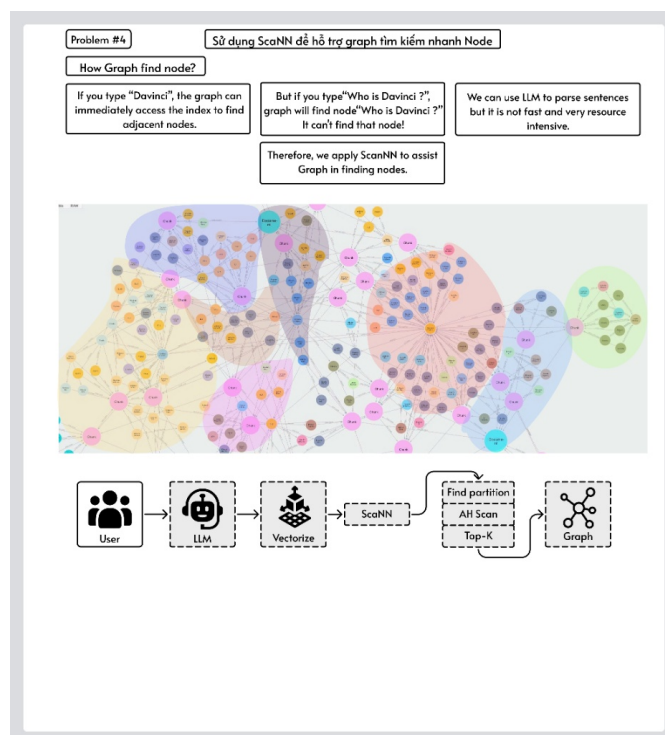


Figure 6: Hybrid Indexing cho phép tìm kiếm mờ cả Text và Entity

```
# Giai đoạn 4: Building Final ScaNN Index (Hybrid)
# Corpus bao gồm cả nội dung văn bản (Chunks) và tên thực thể (Entities)
corpus = [c['content'] for c in all_chunks] + final_entities

# Mã hóa toàn bộ corpus vào cùng không gian vector
all_vectors = embedding_model.encode(corpus)

# Xây dựng bộ tìm kiếm ScaNN hợp nhất
searcher = build_scann_index(all_vectors, num_neighbors=20)

# Khi truy vấn: Tìm cả Text lẫn Entity cùng lúc
idx, dists = searcher.search(query_vec, final_num_neighbors=50)
```

6.5 Vấn đề 5: Truy vết nguồn gốc (Reference Tracking)

Khi hệ thống tìm thấy một thực thể quan trọng (Anchor), cần phải lấy được ngữ cảnh văn bản gốc xung quanh thực thể đó. **Giải pháp:** Nhóm sử dụng kỹ thuật *Context Expansion*. Từ vector của thực thể tìm thấy, hệ thống thực hiện truy vấn ngược lại ScaNN Index để kéo về các đoạn văn bản (Text Chunks) có liên quan nhất, đảm bảo câu trả lời luôn có nguồn gốc rõ ràng.

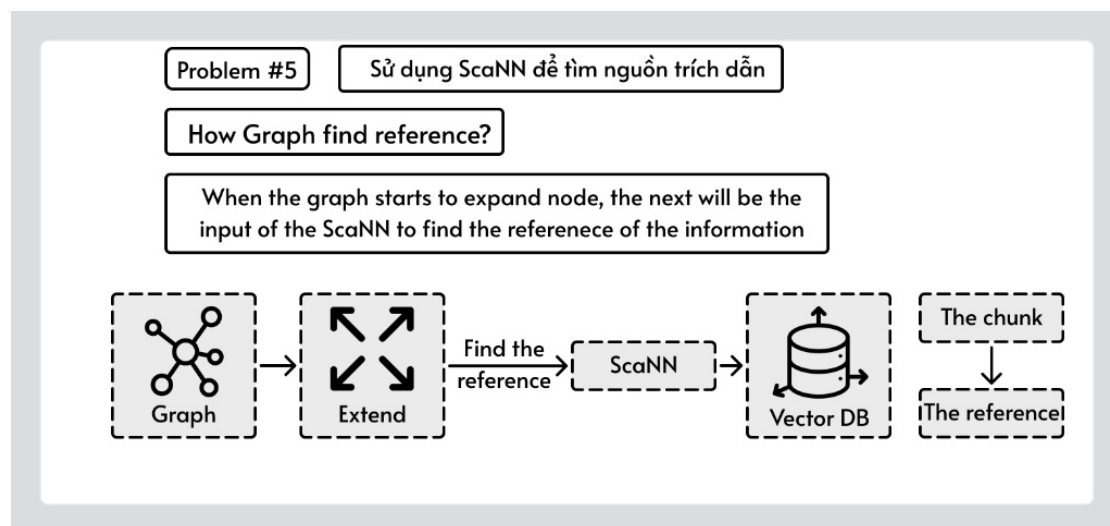


Figure 7: Cơ chế mở rộng ngữ cảnh từ Entity sang Text Chunk

```
# Giai đoạn 5: Retrieval & Expansion
if anchors: # Nếu tìm thấy thực thể (Entity)
    anchor_vec = embedding_model.encode(anchors[0])

    # Tìm kiếm các đoạn văn bản (Chunks) liên quan đến thực thể này
    exp_idx, _ = searcher.search(anchor_vec, final_num_neighbors=5)

    for i in exp_idx:
        item = index_map[i]
        # Bổ sung văn bản gốc vào context cho LLM
        if item['type'] == 'text':
            context.append(f"[Expanded-Rel] {item['content']}")
```

6.6 Vấn đề 6: Bộ nhớ ngắn hạn (Short-term Memory)

Để tránh việc phải suy luận lại các câu hỏi lặp lại hoặc tương tự nhau, hệ thống cần một cơ chế Cache thông minh. **Giải pháp:** Nhóm triển khai kiến trúc **Smart Cache 2 Lớp**. Lớp 1 sử dụng ScaNN để lọc nhanh các câu hỏi cũ có vector tương đồng. Lớp 2 sử dụng mô hình **Cross-Encoder** để kiểm tra kỹ độ trùng khớp ngữ nghĩa. Chỉ khi vượt qua cả 2 lớp kiểm duyệt, câu trả lời cũ mới được trả về.

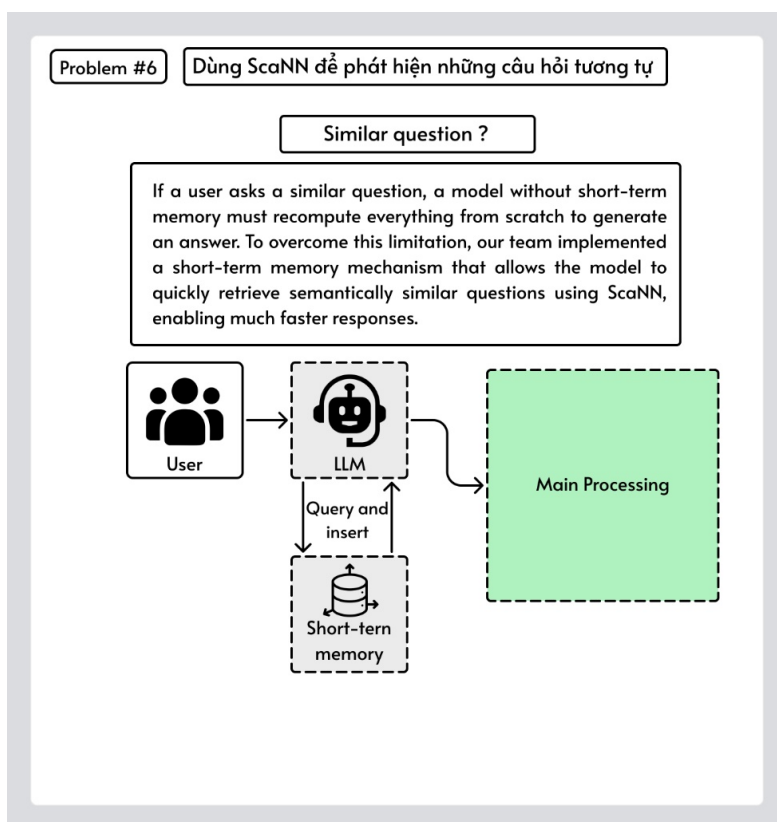


Figure 8: Kiến trúc Smart Cache 2 lớp (ScaNN + Cross-Encoder)

```
# Giai đoạn 6: Smart Semantic Cache
def check_smart_cache(new_query, new_query_vec):
    # LỚP 1: ScaNN (Lọc thô - Nhanh)
    idx, dists = memory_searcher.search(new_query_vec, final_num_neighbors=1)
    if dists[0] < 0.85: return None # Không giống lắm -> Bỏ qua

    # LỚP 2: Cross-Encoder (Kiểm tra tinh - Chính xác cao)
    cached_query_text = memory_data["queries"][idx[0]]

    # Model đánh giá: "Hai câu hỏi này có cùng ý nghĩa không?"
    score = rerank_model.predict([new_query, cached_query_text])

    # Nếu Score > 2.0 (Confidence cao) -> Cache Hit
    if score > 2.0:
        return memory_data["responses"][idx[0]]

    return None
```

Tổng kết: Thông qua việc tích hợp sâu thư viện ScaNN vào cả 6 giai đoạn xử lý (từ Chunking, Cleaning, Indexing đến Caching), hệ thống GraphRAG không chỉ đảm bảo tính toàn vẹn của dữ liệu mà còn đạt được tốc độ truy vấn thời gian thực với độ chính xác cao nhờ cơ chế kiểm định 2 lớp.

7 Đánh giá

7.1 Các tham số chính trong cấu hình ScaNN

Tham số	Ý nghĩa và ảnh hưởng đến hiệu năng
<code>num_leaves</code>	Số lượng cụm (cluster) khi phân vùng không gian vector. Giá trị lớn giúp tăng tốc độ truy vấn (vì mỗi cụm nhỏ hơn) nhưng làm tăng thời gian huấn luyện và bộ nhớ khi xây dựng chỉ mục.
<code>num_leaves_to_search</code>	Số cụm được duyệt khi truy vấn. Giá trị càng lớn thì độ chính xác (recall) càng cao nhưng tốc độ chậm hơn. Giảm giá trị này giúp tăng speed-up.
<code>training_sample_size</code>	Số vector mẫu được dùng để huấn luyện cây phân cụm. Giá trị cao giúp phân cụm chính xác hơn nhưng kéo dài thời gian huấn luyện.
<code>num_subquantizers</code>	Số lượng subquantizer khi lượng tử hoá (quantization). Giá trị cao cho độ chính xác tốt hơn nhưng giảm tốc độ truy vấn.
<code>anisotropic_quantization_threshold</code>	Ngưỡng điều khiển mức nén khi lượng tử hoá. Ngưỡng thấp hơn $\rightarrow$ ít nén hơn, recall cao hơn nhưng tốn bộ nhớ và giảm tốc độ.
<code>reorder(x)</code>	Số lượng điểm ứng viên gần nhất được tính lại chính xác bằng độ tương đồng gốc sau khi tìm kiếm xấp xỉ. Giá trị lớn giúp tăng recall nhưng làm giảm tốc độ truy vấn.

Table 4: Các tham số chính trong cấu hình ScaNN và ảnh hưởng đến hiệu năng.

8 Kết luận

Thông qua quá trình nghiên cứu lý thuyết và triển khai thực nghiệm, đề tài đã hoàn thành các mục tiêu đề ra về việc tìm hiểu và ứng dụng thuật toán ScaNN (Scalable Nearest Neighbors) trong bài toán tìm kiếm vector quy mô lớn.

8.1 Kết quả đạt được

Nhóm thực hiện đã đạt được những kết quả quan trọng sau:

- **Về mặt lý thuyết:** Đã nắm vững và vận dụng thành công kiến trúc cốt lõi của ScaNN bao gồm ba giai đoạn: Phân vùng (Partitioning), Lượng tử hóa vector dị hướng (AVQ) và Sắp xếp lại (Reordering).
- **Về hiệu năng thực nghiệm:**
 - Trên tập thử nghiệm văn bản nhỏ (5 000 vector, 200 truy vấn), ScaNN giảm đáng kể thời gian truy vấn trung bình, khoảng $5.2\text{--}5.5\times$ so với phương pháp brute-force, trong khi $\text{recall}@K$ vẫn duy trì ở mức cao (0.88–0.97). Tuy nhiên, do kích thước dữ liệu nhỏ, ScaNN cần một phần bộ nhớ để xây dựng chỉ mục, dẫn đến mức tiêu thụ bộ nhớ tạm thời cao hơn so với dữ liệu gốc.
 - Mở rộng lên tập dữ liệu lớn $N = 4\,000\,000$ vector (~ 2 GB), ScaNN vẫn vượt trội: thời gian xây dựng ~ 304 s, bộ nhớ ~ 489 MB so với 1.95 GB của brute-force (giảm ~ 4 lần); truy vấn 50 query giảm hơn 80 lần, $\text{recall}@K$ từ 0.79 đến 1.0.

8.2 Hạn chế

Bên cạnh những kết quả đạt được, phương pháp triển khai hiện tại của nhóm vẫn còn tồn tại hạn chế về mặt dữ liệu đầu vào:

- **Giới hạn ngữ nghĩa:** Việc áp dụng ScaNN trong giai đoạn đầu mới chỉ dừng lại ở việc xét tiêu đề (titles) của các bài viết.
- **Thiếu chiều sâu ngữ cảnh:** Do chưa xử lý và vector hóa được toàn bộ nội dung chi tiết (full-text context), hệ thống chưa khai thác được trọn vẹn ngữ nghĩa sâu của văn bản, dẫn đến việc bỏ sót các mối liên hệ tiềm ẩn nằm trong nội dung bài viết.

8.3 Hướng phát triển và Đánh giá phương pháp mở rộng

Để khắc phục hạn chế trên, nhóm đã đề xuất và bước đầu hiện thực hóa kiến trúc **GraphRAG** kết hợp với ScaNN.

- **Cải thiện về mặt lý thuyết:** Kiến trúc mở rộng này được thiết kế để giải quyết các vấn đề ngữ nghĩa thông qua 6 giải pháp cốt lõi như: Phân đoạn văn bản chiến thuật (Strategic Chunking), Hợp nhất thực thể (Entity Disambiguation) và Khai phá tri thức ẩn (Link Prediction). Về mặt lý thuyết, phương pháp này khắc phục được nhược điểm chỉ xét tiêu đề của phương pháp cũ bằng cách xây dựng chỉ mục lai (Hybrid Indexing) giữa văn bản và thực thể đồ thị.



- **Vấn đề về kiểm chứng số liệu:** Tuy nhiên, trong khuôn khổ báo cáo này, hiệu quả của phương pháp mở rộng GraphRAG mới chỉ dừng lại ở mức độ đề xuất kiến trúc và cài đặt chức năng. **Hiệu quả số liệu định lượng** (về độ chính xác, tốc độ cải thiện cụ thể so với phương pháp cũ trên tập dữ liệu lớn) **chưa được tính toán và kiểm chứng cụ thể**. Đây sẽ là nội dung trọng tâm cần được đo lường kỹ lưỡng trong các nghiên cứu tiếp theo.

Mã nguồn và chạy thử:

GitHub repository:

<https://github.com/hoanganh1105/scann-approximate-nearest-neighbor>

Google Colab demo:

Vấn bản : [Colab Notebook 1](#)

Vector : [Colab Notebook 2](#)

Mở rộng: [Colab Notebook 3](#)

Tài liệu tham khảo

- [1] Guo, R., Sun, F., Lindgren, E., Geng, Q., Simcha, D., Chern, F., & Kumar, S. (2020).
Accelerating Large-Scale Inference with Anisotropic Vector Quantization.
In International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research, 2020.
- [2] Malkov, Y. A., & Yashunin, D. A. (2018).
Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs.
IEEE Transactions on Pattern Analysis and Machine Intelligence, 42(4), 824–836.
- [3] Johnson, J., Douze, M., & Jégou, H. (2019).
Billion-Scale Similarity Search with GPUs.
IEEE Transactions on Big Data, 7(3), 535–547.
- [4] Spotify. (2017).
Annoy: Approximate Nearest Neighbors Oh Yeah.
GitHub repository.
- [5] Google Research. (2020).
ScaNN: Efficient Vector Similarity Search.
GitHub repository.
- [6] TensorFlow.
Efficient Serving / TensorFlow Recommenders.
Accessed: Sept. 28, 2025. [Online].
- [7] TensorFlow.
Recommending Movies: Retrieval / TensorFlow Recommenders.
Accessed: Sept. 28, 2025. [Online].
- [8] Google Research.
Announcing ScaNN: Efficient Vector Similarity Search.
Accessed: Sept. 28, 2025. [Online].
- [9] P. Covington, J. Adams, and E. Sargin.
Deep Neural Networks for YouTube Recommendations.
Proceedings of the 10th ACM Conference on Recommender Systems, Boston, MA, USA, Sept. 2016, pp. 191–198.
- [10] S. Raza *et al.*
A Comprehensive Review of Recommender Systems: Transitioning from Theory to Practice.
arXiv preprint arXiv:2407.13699, Feb. 2025.